Week 4 - AWS AppConfig (feature flags, deployment strategies, validators)

DOP-C02 Domain 2 (Configuration Management and IaC). AppConfig is a capability of **Systems Manager**. Typical questions:

1. **Change application behaviour without redeploying code** (feature flag, kill switch, log level, throttling limit).
2. Roll out a **config change gradually** and **roll back automatically** when a CloudWatch alarm fires.
3. **Validate** configuration **before** it reaches the application.
4. Retrieve config **efficiently** from Lambda / ECS / EKS / EC2 = **AppConfig Agent / Lambda extension**.

The big idea: **separate code deploys from config deploys**, and give config the same safety as code: **validation -> gradual rollout -> monitoring -> automatic rollback**.

Part 1 - Core components

```
Application  (e.g. "orders-service")
  Environment: dev    (monitors: CloudWatch alarms)
  Environment: prod   (monitors: 5XX alarm, latency alarm)
  Configuration profile: "feature-flags"  (Feature flags, hosted store, validators)
  Configuration profile: "app-settings"   (Freeform, S3 / Parameter Store / ..., validators)
  Deployment = profile version + environment + deployment strategy
```

Component	Meaning
Application	Logical container (service, app); **extensions** add lifecycle hooks
Environment	Deployment target group (dev, beta, prod, Region, tenant); **monitors = CloudWatch alarms** for automatic rollback
Configuration profile	**Source**, **type** (feature flags / freeform) and **validators**
Deployment strategy	**How fast**: duration, growth type and factor, **bake time**
Deployment	Rolls out **one config version** to **one environment** with **one strategy**

Configuration profile types

Type	Content	Notes
Feature flags (`AWS.AppConfig.FeatureFlags`)	Flags (on / off) with **attributes** and **constraints**, **multi-variant flags** with targeting rules	**Hosted configuration store only**; built-in structure validation; console flag editor
Freeform (`AWS.Freeform`)	Any **JSON, YAML or text**	Any supported source

Configuration sources

Source	Notes
AppConfig hosted configuration store	Simplest; **versioned** in AppConfig; KMS; size quota; **required for feature flags**
Amazon S3	Object (bucket versioning recommended); AppConfig reads it with a **retrieval role**
SSM Parameter Store	String / SecureString parameter
SSM document	Type `ApplicationConfiguration`
Secrets Manager	Secret value
CodePipeline	Config delivered by the pipeline (AppConfig **deploy action**)

- **Retrieval role** (IAM) reads S3 / SSM / Secrets Manager sources; **KMS** CMKs for hosted configs and deployments.

Part 2 - Feature flags

```
{
  "flags": {
    "newCheckout": {
      "name": "New checkout flow",
      "attributes": {
        "maxItems": { "constraints": { "type": "number", "minimum": 1, "maximum": 50 } }
      }
    },
    "killSwitchRecommendations": { "name": "Disable recommendations" }
  },
  "values": {
    "newCheckout": { "enabled": true, "maxItems": 20 },
    "killSwitchRecommendations": { "enabled": false }
  },
  "version": "1"
}
```

Feature	Detail
Basic flag	`enabled: true / false`; toggle in the console, **save a new version, then deploy**
Attributes + constraints	Typed values (number, string, boolean, arrays) with **constraints** (min / max, regex, enum), **validated automatically**
Multi-variant flags	Several **variants** with **rules** evaluated against **context** sent by the client (`userId`, `tier`, `region`); for example beta users get variant B, or percentage splits; **evaluated by the AppConfig Agent**
Short-term flags	Marked temporary with a deprecation date, to avoid flag debt
Use cases	**Feature toggles** (dark launch), **kill switches**, **operational toggles** (log level, cache TTL, rate limits), **allow / deny lists**, A/B-style rollout

CloudWatch Evidently (flags + experiments) is being retired; AWS recommends **AppConfig feature flags** instead.

Part 3 - Validators

Validators run when a deployment starts (and when a hosted version is created). If validation fails, the deployment does not start - invalid config never reaches the application.

Validator	Checks	Notes
JSON Schema	**Syntax / structure**: required fields, types, ranges, enums	Inline JSON Schema on the profile
AWS Lambda	**Semantic / business logic** (URL reachable, discount within margin, IDs exist)	Receives the config (base64) and **throws to reject**; **AppConfig needs invoke permission** (resource policy for `appconfig.amazonaws.com`)
Feature flag built-in	Flag structure + attribute **constraints**	Automatic for the Feature flags type

- A profile can have **both** a JSON Schema and a Lambda validator.
- **Validators** = **before** deployment, check **content**. **Monitors** (alarms) = **during / after** deployment, check the **effect**.

Part 4 - Deployment strategies

Setting	Meaning
Deployment time	Total minutes to reach 100%
Growth type	**Linear** (equal steps) or **Exponential** (small first, then larger)
Growth factor	% of **targets** added each step
Final bake time	Minutes to **keep watching alarms after 100%**; alarm during bake = **rollback**

Targets = the **clients (hosts, containers, functions) that poll** for configuration; the new version goes to a growing **percentage of polling clients**.

Predefined strategies

Strategy	Behaviour	Use
AppConfig.AllAtOnce	**100% immediately**, then bake (about 10 minutes)	Fast, still monitored during bake
AppConfig.Linear50PercentEvery30Seconds	50% every 30 seconds (about 1 minute), short bake	**Testing / demo**
AppConfig.Canary10Percent20Minutes	**Exponential** from 10% over **20 minutes**, about 10-minute bake	**AWS-recommended for production**
AppConfig.Linear20PercentEvery6Minutes	20% every 6 minutes (30 minutes), about 30-minute bake	Production, slow and careful

Custom strategies set your own values (for example Linear 10% every 3 minutes, 30-minute bake).

Deployment behaviour

- **Only one active deployment per environment** at a time.
- **Automatic rollback**: an **environment monitor alarm** in **ALARM** during the deployment **or bake time** = AppConfig **rolls back to the previous version** (`ROLLED_BACK`). Monitors need an **IAM role** with `cloudwatch:DescribeAlarms`.
- `StopDeployment` stops an in-progress deployment and rolls back.
- After completion, **revert by deploying the previous version** (AllAtOnce if urgent).
- States: VALIDATING, DEPLOYING, BAKING, COMPLETE, ROLLING_BACK, ROLLED_BACK.

Part 5 - Retrieving configuration

AppConfig Data API (polling)

```
StartConfigurationSession(app, env, profile, RequiredMinimumPollIntervalInSeconds)
    -> InitialConfigurationToken
GetLatestConfiguration(token)
    -> Configuration (EMPTY if unchanged) + NextPollConfigurationToken + NextPollIntervalInSeconds
```

- **Always use the new token** for the next call; **empty body = no change** (keep the cache).
- IAM: `appconfig:StartConfigurationSession` + `appconfig:GetLatestConfiguration`.
- Old `GetConfiguration` **is deprecated**.
- Billed per **configuration request received**: cache and poll sensibly.

AppConfig Agent (recommended)

Where	How
Lambda	**AWS AppConfig Lambda extension** (layer); read `http://localhost:2772/applications/{app}/environments/{env}/configurations/{profile}`; caches and polls in the background
ECS / EKS	**AppConfig Agent sidecar container** (same localhost endpoint)
EC2 / on-prem	**AppConfig Agent** service

Benefits: **lower latency and cost** (cache), **background polling**, **multi-variant flag evaluation** with context, write config to disk, token handling.

"Lambda reads AppConfig on every invocation, high latency and cost" = **AppConfig Lambda extension**.

Part 6 - Extensions

Action points: `PRE_CREATE_HOSTED_CONFIGURATION_VERSION`, `PRE_START_DEPLOYMENT`, `ON_DEPLOYMENT_START`, `ON_DEPLOYMENT_STEP`, `ON_DEPLOYMENT_BAKING`, `ON_DEPLOYMENT_COMPLETE`, `ON_DEPLOYMENT_ROLLED_BACK`.

Extension	Use
AWS-authored notification extensions	Deployment events to **EventBridge**, **SNS**, **SQS** (for example notify Slack on rollback)
Custom (Lambda)	**PRE_START_DEPLOYMENT** to block deploys in a freeze or require a ticket, back up config, extra checks
Jira / integrations	Track flag changes in tickets

Extensions are **associated** with an application, environment or profile.

Part 7 - CI/CD, IaC and comparisons

- **CodePipeline AppConfig deploy action**: config files from the pipeline source into a profile + environment with a strategy (config-as-code with approvals).
- **CloudFormation**: `AWS::AppConfig::Application`, `Environment` (Monitors), `ConfigurationProfile` (Validators), `HostedConfigurationVersion`, `DeploymentStrategy`, `Deployment`, `Extension`, `ExtensionAssociation`; CDK `aws-appconfig`.
- **Separation of duties**: code through CodePipeline, flags toggled in AppConfig, audited in **CloudTrail**; IAM restricts `StartDeployment` on prod.

Need	Best choice
Gradual rollout + validation + automatic rollback for config / flags	**AppConfig**
Feature flags / kill switch without redeploying	**AppConfig feature flags**
Simple key / value config, hierarchy, latest AMI	**Parameter Store** (no validation, rollout or rollback)
Secrets with **rotation**	**Secrets Manager** (can be an AppConfig source)
Lambda env vars / ECS task env	Changing them **needs a function / task update** - not dynamic

Part 8 - Exam traps

Trap	Truth
Redeploy Lambda / ECS to change a flag	**AppConfig**: no code deploy
Feature flags stored in S3	**Feature flag profiles use the hosted store**
Validation happens after deployment	**Before**; failure = no deployment
Lambda validator AccessDenied	**AppConfig needs invoke permission** (resource policy)
JSON Schema checks business rules	Schema = structure; **logic = Lambda validator**
Automatic rollback without alarms	Needs **environment monitors (alarms) + role**
Rollback only during rollout	**Also during final bake time**
Stop a bad deployment now	`StopDeployment`
Two deployments to one environment	**One active deployment per environment**
Production canary for config	**Canary10Percent20Minutes** (or custom exponential)
Fastest rollout, still monitored	**AllAtOnce** (has bake time)
Percentage = % of users	% of **polling targets**; per-user = **multi-variant flags + context**
Lambda calls AppConfig every invocation	**Lambda extension** (`localhost:2772`)
ECS / EKS retrieval	**Agent sidecar**
Use GetConfiguration	Deprecated; **StartConfigurationSession + GetLatestConfiguration**
Empty GetLatestConfiguration = error	**No change**; keep cache
Notify team on rollback	**Extension** to EventBridge / SNS
Block deploys during a freeze	**PRE_START_DEPLOYMENT extension**
Evidently for new feature flags	Retiring; **AppConfig feature flags**
Parameter Store gradual rollout	**No**; AppConfig can use it as a source

Part 9 - Hands-on lab (about 45 to 60 minutes, very low cost)

Lab 1 - Application, environment, feature flags

1. Create application `orders-app` and environment `prod` (no monitors yet).
2. Create a **Feature flags** profile `flags`: flag `newCheckout` (disabled) with attribute `maxItems` (number, min 1, max 50, value 20) and flag `killSwitchRecs` (disabled). Save version 1.
3. Set `maxItems = 100` and save: **rejected by the constraint**. Set it back to 20.

Lab 2 - Deployment strategy and deploy

1. Custom strategy `Linear25Every1Min`: Linear, 25%, 4 minutes, **bake 2 minutes**.
2. Deploy `flags` v1 to `prod`: watch DEPLOYING, BAKING, COMPLETE.
3. Start a second deployment while the first runs: **rejected**.

Lab 3 - Read config from Lambda

1. Python Lambda with the **AppConfig Lambda extension layer**; role with `appconfig:StartConfigurationSession` and `appconfig:GetLatestConfiguration`:

```
import urllib.request, json
def handler(e, c):
    url = ("http://localhost:2772/applications/orders-app"
           "/environments/prod/configurations/flags")
    flags = json.loads(urllib.request.urlopen(url).read())
    return {"newCheckout": flags["newCheckout"]["enabled"]}
```

2. Invoke: `false`. Enable `newCheckout`, save **v2**, deploy with **AllAtOnce**, invoke again after the poll interval: `true` - **no code change**.

Lab 4 - Validators

1. Freeform profile `settings` (hosted): `{"logLevel":"INFO","timeoutMs":3000}`; **JSON Schema validator**: `logLevel` enum DEBUG / INFO / WARN / ERROR, `timeoutMs` integer 100 to 10000.
2. Version `{"logLevel":"VERBOSE"}` and deploy: **validation fails, nothing deployed**.
3. Optional: **Lambda validator** rejecting `timeoutMs > 5000`, with a resource policy for `appconfig.amazonaws.com`; test 6000.

Lab 5 - Automatic rollback and notifications

1. CloudWatch alarm on custom metric `Demo/Errors` (>= 1, 1 datapoint, 1 minute).
2. Add it as a **monitor** on `prod` with a role allowing `cloudwatch:DescribeAlarms`.
3. Deploy a new version; during it run `aws cloudwatch put-metric-data --namespace Demo --metric-name Errors --value 5`: **ROLLED_BACK**.
4. Associate the **AppConfig deployment events to SNS** (or EventBridge) extension; repeat and receive the **rollback notification**.
5. Start another deployment and run `aws appconfig stop-deployment ...`: rolled back.

Cleanup: delete environment, profiles and hosted versions, strategy, extension associations, Lambda, alarm, SNS topic.

Part 10 - Check yourself

Questions

1. What problem does AppConfig solve compared with Lambda environment variables?
2. The five main AppConfig components?
3. Supported sources, and which one feature flags must use?
4. JSON Schema vs Lambda validator - what each checks, and when validators run?
5. A Lambda validator never runs and the deployment fails with an access error. Why?
6. Deployment time, growth type, growth factor, final bake time?
7. Which predefined strategy for production, and which for testing?
8. How does automatic rollback work, and when can it trigger?
9. Stop a bad deployment immediately? Revert after completion?
10. Can two deployments run in the same environment at once?
11. The two AppConfig Data API calls, and what an empty response means?
12. Lambda reads AppConfig every invocation, high latency and cost. Fix?
13. How do ECS / EKS containers retrieve AppConfig efficiently?
14. Slack / SNS notification when a config deployment rolls back?
15. Beta customers get variant B, everyone else variant A - which feature?

Answers

1. Config changes **without redeploying code**, with **validation, gradual rollout, monitoring, automatic rollback**; env vars need a function update.
2. **Application**, **environment** (monitors), **configuration profile** (source, type, validators), **deployment strategy**, **deployment** (+ extensions).
3. Hosted store, S3, Parameter Store, SSM document, Secrets Manager, CodePipeline; feature flags = **hosted store**.
4. **Schema** = structure / types; **Lambda** = business logic; both run **before** deployment, failure = no deployment.
5. AppConfig lacks **invoke permission**; add a Lambda resource policy for appconfig.amazonaws.com.
6. Total duration; linear or exponential steps; % of targets per step; **monitoring minutes after 100%**.
7. **Canary10Percent20Minutes** (or Linear20PercentEvery6Minutes) for production; **Linear50PercentEvery30Seconds** for testing.
8. Environment **monitor alarm** (via IAM role) in ALARM during deployment **or bake time** = rollback to previous version.
9. `StopDeployment`; after completion **deploy the previous version**.
10. **No**, one active deployment per environment.
11. `StartConfigurationSession` and `GetLatestConfiguration` (next token each time); empty = **no change**.
12. **AppConfig Lambda extension** (`localhost:2772`, cache, background polling).
13. **AppConfig Agent sidecar**.
14. **Extension** (AWS-authored SNS / EventBridge) on **ON_DEPLOYMENT_ROLLED_BACK**.
15. **Multi-variant feature flags** with targeting rules on context.

Quick memory summary

- **AppConfig** (Systems Manager) = safe **dynamic config and feature flags without redeploying code**.
- **Application -> Environment (alarm monitors) -> Configuration profile (Feature flags = hosted store; Freeform from hosted / S3 / Parameter Store / SSM doc / Secrets Manager / CodePipeline) -> Deployment (version + environment + strategy)**.
- **Validators before deploy**: JSON Schema (structure) + Lambda (logic, invoke permission); flags have built-in constraints.
- **Strategies**: duration, linear / exponential, growth %, **final bake time**; AllAtOnce, Linear50PercentEvery30Seconds (test), **Canary10Percent20Minutes** (prod), Linear20PercentEvery6Minutes.
- **Automatic rollback** on alarm during deploy or bake; **StopDeployment**; one deployment per environment.
- **Retrieval**: StartConfigurationSession + GetLatestConfiguration, or the **AppConfig Agent** (Lambda extension localhost:2772, ECS / EKS sidecar, EC2).
- **Extensions**: EventBridge / SNS / SQS, PRE_START_DEPLOYMENT checks; **Evidently retiring - use AppConfig flags**.